

Ask the Sensors

Grounded, Explainable Activity Question Answering from Wearable Signals

CS60055 Ubiquitous Computing - Hackathon Challenge 1

Preliminary implementation report - 11 September 2026

PRELIMINARY STATUS NOTICE: This revision documents the implemented repository scaffold and preprocessing pipeline, together with the planned recognition, question-answering, and evaluation design. Recognition, real-data evaluation, and small-language-model experiments are still in progress. Every unavailable result is marked explicitly; no synthetic quantity is presented as a measured result.

Team and contribution statement

Member	Contribution recorded in this revision
23CS30041 - Pritam Mondal	Repository integration, design review, implementation verification, report preparation, and planned end-to-end integration.
MEMBER_2_ROLL - MEMBER_2_NAME	REPLACE BEFORE FINAL: contribution information has not yet been supplied.
MEMBER_3_ROLL - MEMBER_3_NAME	REPLACE BEFORE FINAL: contribution information has not yet been supplied.

Submission snapshot

This report accompanies the public source repository <https://github.com/mpritam17/ask-the-sensors>. The repository is intended to remain reproducible: raw ExtraSensory data and downloaded language-model weights are excluded, while download, preprocessing, training, evaluation, and inference entry points are kept in source control.

Document status

- Implemented and tested: configuration, raw-file parsing, label mapping, resampling, missing-data validity masks, windowing, synthetic data generation, and dataset assembly.
- Verified: 20 committed tests passed in an isolated Python environment.
- In progress: feature extraction, activity recognition, activity timeline construction, Tasks 1-4, real-data metrics, efficiency measurements, and the five required figures.

1. Abstract and Problem Formulation

Wearable accelerometers and gyroscopes provide continuous evidence about motion, but their raw samples are not directly useful to a caregiver or clinician. A conventional activity classifier also leaves an important gap: a sequence of labels does not by itself answer how long an activity lasted, how often it occurred, when it began, or why the recorded signal supports the conclusion. The challenge therefore asks for a sensor question-answering service that converts a natural-language query and a multimodal recording into a direct answer with traceable evidence.

The proposed system separates signal computation from language generation. Sensor processing identifies activity evidence in 2.56-second windows. A timeline layer turns those predictions into intervals, durations, counts, and transitions. Deterministic operations answer identification, verification, duration, count, onset, and comparison questions. A small language model is reserved for open-world questions, and even there it receives only validated interval and feature summaries. A final validator prevents unsupported timestamps or modalities from entering the response.

1.1 Required sensor and activity space

The input contains a timestamped triaxial accelerometer and triaxial gyroscope. Both streams are resampled to 25 Hz. The fixed recognition vocabulary is: lying down, sitting, standing in place, standing and moving, walking, running, and bicycling. These classes are imbalanced and some are difficult to distinguish using a freely oriented phone, which motivates user-disjoint evaluation and macro-averaged metrics.

1.2 Four question tiers

1. **Activity identification:** open identification and binary verification.
2. **Temporal and quantitative reasoning:** duration, count, occurrence time, and activity comparison.
3. **Evidence grounding:** answer correctness plus the supporting interval, modality, channels, and signal-based reasoning.
4. **Open-world reasoning:** behavior-level interpretations beyond a fixed class name, while remaining grounded in observed evidence.

1.3 Uniform answer contract

Answer: <direct answer to the query, or N/A>
Activity/Event: <activity or event, or N/A>
Evidence:
 Timestamp(s): <time range or ranges, or N/A>
 Sensor Modality: <accelerometer, gyroscope, both, or N/A>
 Sensor Channel(s): <Acc X/Y/Z, Gyro X/Y/Z, All, or N/A>
Explanation: <reasoning grounded in the observed signal, or N/A>

All emitted timestamps use seconds from the start of the retained recording. An absolute Unix origin is stored in processed data only to permit later conversion to clock time.

2. Dataset and Data Challenges

Development uses ExtraSensory, a naturalistic dataset collected from sixty users carrying smartphones and wearing smartwatches during daily life [1,2]. This challenge restricts the input modalities to accelerometer and gyroscope. Unlike a laboratory activity dataset, ExtraSensory contains irregular sampling, missing sensor sessions, user-dependent device placement, overlapping self-reports, and a strongly skewed activity distribution. These are treated as model and evaluation constraints rather than removed silently.

2.1 Observation structure

ExtraSensory phone sensors are organized as short sessions associated with per-user example timestamps. The current design assumes approximately 20 seconds of samples per observed session and a nominal source rate near 40 Hz. Successive examples may be separated by unobserved time. The system therefore does not interpolate across inter-session gaps. Duration is accumulated only over supported intervals, and a timeline gap remains a gap rather than manufactured evidence.

2.2 Label source

The seven challenge activities correspond to the original ExtraSensory main-activity choices. The cleaned release collapses `STANDING_IN_PLACE` and `STANDING_AND_MOVING` into `OR_standing`, so it cannot alone preserve the required seven-way target. The pipeline therefore reads the original per-user labels and uses cleaned labels as a consistency check. Examples with no main activity, multiple main activities, or an explicit cleaned-label contradiction are excluded. Unknown cleaned values are tolerated instead of being interpreted as negatives.

2.3 Imbalance and generalization

Sedentary classes dominate the dataset, while running and bicycling are rare. The planned recognition layer uses balanced class weights and caps the number of windows contributed by one user and class. Train, validation, recognition-test, and end-to-end QA sets are separated by user. This prevents overlapping windows from the same session, or idiosyncratic gait and phone placement from one person, leaking across partitions.

2.4 Verification obligations

The large raw archives were not present when this preliminary report was generated. Before reporting real results, the team will run the layout inspection script and record: archive nesting, file suffixes, column count, whether timestamps are explicit, measured sampling rate, accelerometer units, missing-modality frequency, rescaling counts, and per-class label-filter drops.

Item	Preliminary status
Original and cleaned label logic	Implemented; real files pending inspection
25 Hz resampling and missing-data mask	Implemented; synthetic tests passed
Full raw accelerometer and gyroscope build	PENDING REAL-DATA EXPERIMENT
User-disjoint class distribution	PENDING REAL-DATA EXPERIMENT

3. System Architecture

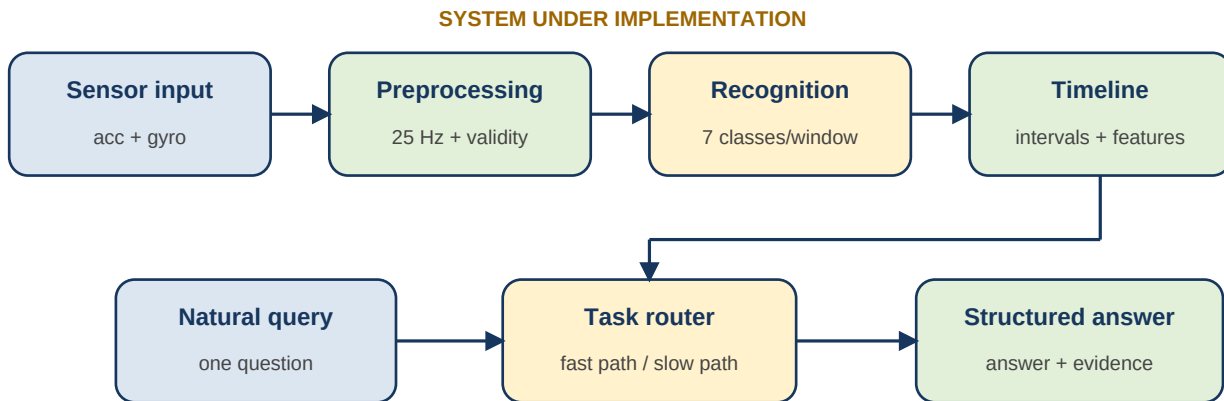


Figure 1. Planned two-speed architecture. Components with implemented preprocessing code are shown first; recognition, timeline, deterministic QA, and the validated SLM path are under implementation in this preliminary revision.

3.1 Two-speed reasoning

Tasks 1-3 are structured computations over an activity timeline. For example, walking duration is a sum of supported walking intervals, and a running onset is the start of the first supported running interval. Delegating such operations to a generative model would make exact arithmetic and evidence alignment less reliable. The proposed fast path therefore uses deterministic query routing and timeline operations.

Task 4 requires interpretation of broader behaviors that may not have a fixed classifier label. The slow path supplies a small language model with a serialized timeline, measured signal features, and an allow-list of evidence intervals. Generated output is accepted only after schema and grounding validation. A rule-based fallback handles a small set of safety-relevant patterns when model loading fails or generation is unsupported.

3.2 Grounding invariant

The interval used to calculate an answer is also the interval emitted as evidence. This eliminates a common failure mode in which an explanation cites a different time span from the one used by the computation. Evidence channel names come from feature provenance. Static posture explanations emphasize accelerometer orientation and variance; gait explanations combine acceleration cadence and gyroscope motion; unsteadiness reasoning emphasizes irregular gyroscope energy.

3.3 Failure behavior

Low-coverage windows are excluded rather than guessed. Low-confidence predictions become unknown. If a question cannot be routed safely, the system returns *Inconclusive* or *N/A* in the required schema. The language model cannot introduce an interval not present in its evidence input, and malformed structured output is rejected.

4. Implemented Preprocessing

The repository currently implements a reproducible preprocessing path from ExtraSensory-style files to compressed per-user window arrays.

4.1 Raw input handling

The raw reader recursively discovers files by sensor-name fragments and timestamp-bearing filenames. It accepts either `x, y, z` files with an implied nominal time axis or files containing `t, x, y, z`. Absolute or relative timestamps are normalized to the start of each session. Empty, truncated, and non-finite files are treated as unavailable sensor observations.

4.2 Irregular resampling and validity

Samples are sorted and duplicate timestamps are removed. Each channel is linearly interpolated on a 25 Hz grid, but interpolation values are not automatically trusted. A grid sample is valid only when a real input sample lies within 0.08 seconds. Accelerometer and gyroscope streams are aligned into the fixed order Acc X, Acc Y, Acc Z, Gyro X, Gyro Y, Gyro Z; combined validity requires support from both modalities.

4.3 Units and windows

Accelerometer magnitude is inspected per example. Values that resemble metres per second squared are divided by standard gravity, producing a consistent gravity-unit representation. Twenty-second sessions yield fourteen 64-sample windows with a 32-sample hop. Each window stores start/end time, validity fraction, example timestamp, class index, and the six signal channels.

4.4 Synthetic reproducibility

The synthetic generator creates seven physically motivated placeholder profiles with sampling jitter, dropouts, missing examples, class imbalance, and the same file organization expected by the reader. It exists only to test data flow. Synthetic outputs are not evidence of real-world accuracy.

4.5 Verified tests and known corrections

An isolated run of the committed suite completed with **20 tests passed**. The tests cover label normalization and filtering, irregular resampling, dropout masks, multimodal alignment, window counts and timestamps, and a synthetic raw-to-window build.

Two issues were identified during audit and are being corrected: a truly empty timestamp array reaches an indexing operation before the empty-input return, and the recording origin is currently chosen from the earliest retained label rather than the first sensor example that survives coverage filtering. Neither issue is hidden in the preliminary status.

5. Recognition Design

The recognition backbone will use engineered features with compact classical models. This choice supports fast CPU inference, direct feature provenance, and several accuracy-versus-overhead operating points without a long neural-training cycle.

5.1 Feature groups

- **Statistical:** mean, standard deviation, median, interquartile range, minimum, maximum, root-mean-square value, energy, skewness, and kurtosis per axis and magnitude.
- **Temporal:** demeaned zero-crossing rate and cross-axis correlations.
- **Spectral:** dominant non-DC frequency, spectral entropy, and power in 0.3-3 Hz, 3-8 Hz, and 8-12 Hz bands.
- **Provenance:** every feature retains its source modality and channel group for later evidence selection.

5.2 Candidate models

Configuration	Purpose	Status
Random forest, 300 trees, depth 18	Primary accuracy-oriented operating point	PENDING REAL-DATA EXPERIMENT
Random forest, 80 trees, depth 12	Compact latency/size operating point	PENDING REAL-DATA EXPERIMENT
Balanced multinomial logistic regression	Interpretable low-cost baseline	PENDING REAL-DATA EXPERIMENT

All candidates use class balancing. The selected recognizer maximizes validation macro-F1; if two models differ by no more than one percentage point, the smaller and faster model is selected. The untouched test users are used once after model selection.

5.3 Stored artifact

The final artifact will include the fitted estimator, any required scaler, feature schema and order, seven-class order, split identifiers, preprocessing configuration hash, and training seed. The artifact must stay below GitHub's individual-file size limit so evaluation can run without retraining.

5.4 Preliminary accuracy table

Metric	Value
Test accuracy	PENDING REAL-DATA EXPERIMENT
Balanced accuracy	PENDING REAL-DATA EXPERIMENT
Macro-F1	PENDING REAL-DATA EXPERIMENT
Per-class precision/recall/F1	PENDING REAL-DATA EXPERIMENT

No recognition accuracy is claimed in this preliminary revision.

6. Timeline and Tasks 1-3

Window probabilities will be smoothed with a short median/majority filter. Windows whose maximum probability is below 0.35 are marked unknown. Consecutive supported windows are aggregated into intervals containing the predicted activity, start/end time, observed duration, mean confidence, contributing windows, and representative features. Intervals are split across missing-data gaps; isolated single-window changes are absorbed only when both neighbors agree.

6.1 Deterministic operations

Question type	Timeline operation
Identification	Highest-duration supported activity in the requested span
Verification	Presence of a supported interval for the named activity
Duration	Sum of supported interval durations
Count	Number of separated intervals
Onset/when	Start of the first matching interval
Comparison	Compare summed durations of the two named activities

6.2 Evidence selection

Every operation returns the intervals it consumed. Those intervals populate the timestamp field. Feature provenance selects modalities and channels: posture uses acceleration orientation and stability, walking/running combine cadence and gyroscope oscillation, and bicycling uses smooth cadence with sustained gyroscope motion. Explanations include measured values and confidence rather than stock descriptions alone.

6.3 Illustrative output format

The following block illustrates formatting only; it is not a measured prediction.

```
Answer: Yes, running began at 1512 seconds
Activity/Event: Onset of running
Evidence:
  Timestamp(s): 1512 to 1980 (seconds from start)
  Sensor Modality: Both
  Sensor Channel(s): All
Explanation: Illustrative example: the cited interval would need to show a
              sustained high accelerometer cadence and gyroscope energy.
```

6.4 Interface

The final command accepts a CSV with `timestamp`, `acc_x`, `acc_y`, `acc_z`, `gyro_x`, `gyro_y`, `gyro_z`, a text file containing one question per line, and an output filename. It writes one schema-conforming block per question. Input validation failures return a non-zero exit; insufficient signal returns a structured N/A answer rather than a crash.

7. Task 4: Grounded Open-World Reasoning

The planned slow path uses Qwen2.5-1.5B-Instruct [3], selected as a compact instruction-tuned model that can fit on the available 8 GB laptop GPU. This component is not yet integrated, and no Task 4 score is claimed in this revision.

7.1 Restricted evidence prompt

The model receives the query, activity intervals, observed durations, missing-data gaps, per-interval feature summaries, and an explicit allow-list of evidence ranges. Raw signal arrays are not converted into free text. Generation is deterministic: temperature zero, greedy decoding, and a small output-token budget.

7.2 Structured validation

The model must return JSON containing answer, activity/event, timestamps, modalities, channels, and explanation. A deterministic validator rejects malformed output, timestamps outside supplied evidence, nonexistent modalities, unsupported channel names, and explanations that refer to absent features. Only validated fields are formatted for the user.

7.3 Fallback rules

- Prolonged rest requires a sufficiently long supported lying interval.
- Wheeled or pedal-based movement requires a supported bicycling interval and compatible smooth cadence.
- Strenuous activity requires supported high-intensity activity, normally running.
- Possible unsteadiness requires irregular gyroscope energy outside a normal running/cycling cadence.

The rule layer also serves when model weights are unavailable. A query outside both the validated model result and rule coverage returns Inconclusive with an explanation of the evidence limitation.

7.4 Privacy and cost

All inference is intended to remain local. The language model reads only the derived evidence summary, reducing prompt length and keeping raw health-related motion data off external services. Tasks 1-3 do not invoke the model, so their latency and memory cost remain close to that of the compact recognition pipeline.

Task 4 item	Preliminary status
Local model loading	PENDING IMPLEMENTATION
JSON/schema validator	PENDING IMPLEMENTATION
Timestamp containment tests	PENDING IMPLEMENTATION
Open-world accuracy and rubric score	PENDING REAL-DATA EXPERIMENT

8. Evaluation Protocol

Evaluation is performed on users excluded from training and model selection. A balanced question set covers identification, verification, duration, count, comparison, grounding, and open-world reasoning. Template-generated questions are supplemented with paraphrases so the router is not assessed solely on wording generated by its own code.

8.1 Metrics

- **Categorical:** exact-match accuracy, macro-F1, and balanced accuracy.
- **Binary verification:** positive-class precision, recall, and F1, plus specificity.
- **Numeric:** mean absolute error and accuracy within declared absolute or relative tolerances.
- **Temporal:** interval Intersection over Union, temporal precision/recall/F1, and acceptance from IoU 0.1 to 0.9.
- **Grounding:** an answer is grounded-correct only when the answer is correct, evidence IoU is at least 0.5, and modality/channels match the reference.
- **Open world:** categorical behavior correctness plus a fixed 1-5 faithfulness/plausibility rubric. Independent human agreement will be reported only if independent grading actually occurs.

8.2 Headline score

Overall QA accuracy is macro-averaged across question types, preventing abundant sedentary identification examples from dominating temporal or open-world questions.

Figure 2 - Accuracy by question type

Pending real-data experiment — no result available in this preliminary revision.

Figure 3 - Seven-class activity confusion matrix

Pending real-data experiment — no result available in this preliminary revision.

Both panels above are reserved in this preliminary report so the final figure numbering and discussion remain stable.

9. Required Figures and Efficiency Plan

Figure 4 - Accuracy versus strictness

Pending real-data experiment — no result available in this preliminary revision.

Figure 5 - Accuracy versus overhead and Pareto frontier

Pending real-data experiment — no result available in this preliminary revision.

Figure 6 - Robustness versus dropped input samples

Pending real-data experiment — no result available in this preliminary revision.

Each placeholder states the absence of results rather than displaying invented axes or values. The final robustness experiment will randomly drop 0, 5, 10, 20, and 30 percent of input samples using fixed seeds, then run the unchanged preprocessing and QA stack.

9.1 Efficiency target

Measurements will be performed on an AMD Ryzen 9 8940HX laptop with an NVIDIA GeForce RTX 5050 Laptop GPU (8 GB VRAM). The report will identify the operating system, Python environment, model configuration, and run count.

9.2 Measurements

- Recognizer and language-model parameter counts and on-disk sizes.
- Peak inference memory for deterministic and SLM paths.
- Median and 95th-percentile latency over at least 30 warmed-up single-query runs.
- CPU/GPU utilization where measurement is reliable.
- Separate Tasks 1-3 and Task 4 results, because only the latter invokes Qwen.

9.3 Accuracy-overhead operating points

The logistic regression, compact random forest, and full random forest form three genuine recognition operating points. Each is evaluated with the same timeline, questions, and scoring code. Non-dominated points are joined to form the required Pareto frontier. Quantization or pruning is attempted only after the core evaluation is stable.

9.4 Current limitations

This preliminary revision has no raw ExtraSensory build, trained recognizer, QA output, SLM execution, real accuracy, robustness curve, or measured latency. Its evidence is limited to source inspection and 20 passing preprocessing tests. These limitations will be replaced by measured results, not silently removed, in the final revision.

10. Reproducibility, Integrity, and References

10.1 Current setup and verification

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -e .
pytest -q
```

Synthetic preprocessing smoke test:

```
python scripts/make_synthetic_data.py --users 3
python scripts/build_dataset.py --raw-root data/raw/synthetic --synthetic
pytest -q
```

The final README will add commands for data download, training, evaluation, model retrieval, question answering, benchmarking, and report regeneration. Raw data and large language-model weights remain excluded from Git.

10.2 Academic integrity and AI-use disclosure

Claude (Anthropic) was used as a pair-programming and design assistant for the repository scaffold, configuration, preprocessing modules, synthetic generator, and the initial unit-test suite. The team is responsible for reviewing and validating that work against real data.

OpenAI Codex was used to audit the assignment against the repository, identify missing deliverables and preprocessing defects, plan the staged implementation, verify the existing test suite, initialize repository history, and draft/build this preliminary report. Future Codex-assisted implementation and testing will be recorded in docs/AI_USE_LOG.md. AI-generated text and code are not treated as evidence of correctness; reported results must come from reproducible commands and saved outputs.

10.3 References

- [1] ExtraSensory Dataset, UC San Diego. <http://extrasensory.ucsd.edu/>
- [2] Y. Vaizman, K. Ellis, and G. Lanckriet. ExtraSensory naturalistic context-recognition dataset and study. IEEE Pervasive Computing, 16(4), 2017.
- [3] Qwen Team. Qwen2.5-1.5B-Instruct model card. <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>
- [4] C. R. Harris et al. Array programming with NumPy. Nature, 585, 2020. <https://doi.org/10.1038/s41586-020-2649-2>
- [5] P. Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17, 2020. <https://doi.org/10.1038/s41592-019-0686-2>
- [6] F. Pedregosa et al. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2011.
- [7] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. NeurIPS, 2019.
- [8] T. Wolf et al. Transformers: State-of-the-art natural language processing. EMNLP System Demonstrations, 2020.

10.4 Revision policy

This PDF is intentionally versioned as a preliminary submission. Later revisions will preserve the methodology and replace pending panels with traceable metrics and figures. The Git commit history records when each result became available.